

StudentIQ

Predicting Final Exam Marks Using Machine Learning Regression

Project Documentation

Domain: Student Academic Performance Prediction
Approach: Regression (Linear & Polynomial Regression)
Deployment: Streamlit Web Application

Group 1: AITookMyJob

Mahesh Rajpurohit

Kirthish Shetty

Sudarshan Devadiga

Abdullah Haque

Fauzan Baig

1. Introduction

Academic performance prediction is one of the most practical applications of regression analysis in the education sector. Instructors and institutions often want to know, well before the final examination, which students are likely to underperform so that timely intervention — extra tutoring, counselling, or attendance follow-up — can be arranged. This project builds a complete, end-to-end machine learning pipeline that predicts a student's Final Exam Marks (out of 100) using five academic and behavioural indicators: attendance percentage, two internal test scores, an assignment score, and daily study hours. The project follows the standard data science lifecycle: data cleaning, exploratory data analysis, feature scaling, model training, rigorous evaluation, interpretation of results, and finally deployment as an interactive Streamlit web application that anyone — a teacher, mentor, or the student themselves — can use to obtain an instant, data-driven prediction.

The problem is framed strictly as a regression task because the target variable, Final Exam Marks, is continuous (ranging from 0 to 100) rather than a discrete category. Two regression techniques were implemented and compared: (1) Linear Regression, which models the final marks as a weighted linear combination of the input features, and (2) Polynomial Regression of degree 2, which additionally captures non-linear relationships and pairwise interaction effects between features (for example, the combined effect of attendance and study hours together, which may be more than the sum of their individual effects). Both models were trained on identical, properly scaled data and evaluated using four industry-standard regression metrics — Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and the Coefficient of Determination (R^2) — to allow a fair, quantitative comparison.

2. Dataset Description

The dataset used for this project, `Final_Marks_Data.csv`, contains 2,000 individual student records collected across seven columns. Each row represents one student, uniquely identified by a `Student_ID` field (values such as S1000, S1001, and so on), which carries no predictive information about academic performance and is therefore excluded from the modelling features. The remaining six columns consist of five predictor (independent) variables and one target (dependent) variable, described in the table below.

Column	Description
Student_ID	Unique alphanumeric identifier for each student (e.g., S1000). Dropped before modelling — it is an identifier, not a predictive feature.
Attendance (%)	Percentage of classes the student attended during the term (0–100).
Internal Test 1 (out of 40)	Marks scored in the first internal / mid-term assessment.
Internal Test 2 (out of 40)	Marks scored in the second internal / mid-term assessment.
Assignment Score (out of 10)	Marks awarded for assignment submissions and quality.

Column	Description
Daily Study Hours	Average number of hours the student reports studying per day.
Final Exam Marks (out of 100)	TARGET VARIABLE — the score the model is trained to predict.

A preliminary data quality check confirmed that the dataset is exceptionally clean: there are zero missing (null) values across all 2,000 rows and seven columns, and zero duplicate rows. All six modelling-relevant columns are already numeric (integer type), so no categorical encoding (such as one-hot encoding or label encoding) was necessary for this particular dataset — the only preprocessing required was feature scaling, which is covered in detail in Section 4.

3. Detailed Walkthrough of the Jupyter Notebook (Student_Final_Marks_Prediction.ipynb)

This section explains, block by block, every single code cell in the accompanying Jupyter notebook. Each subsection reproduces the exact code from that cell and follows it with a detailed paragraph explaining what the code does, why it is necessary at that stage of the pipeline, and how it connects to the overall goal of building an accurate, interpretable, and deployable regression model.

3.1 Importing Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import joblib
sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (8, 5)
```

This first cell imports every library used throughout the notebook and configures the default plotting style. NumPy and pandas form the numerical and tabular-data backbone of the entire pipeline: pandas is used to load the CSV file into a DataFrame and to perform all row/column-level operations (dropping columns, splitting into features and target, inspecting statistics), while NumPy underlies pandas internally and is used directly for numerical operations such as computing the square root for RMSE. Matplotlib and Seaborn are the visualization libraries — Matplotlib provides the low-level figure and axis objects, while Seaborn is built on top of it to produce more polished statistical plots (heatmaps, histograms with density curves, scatter plots) with far less code. From scikit-learn, four specific submodules are imported: `train_test_split` from `model_selection`, which is used to randomly partition the dataset into training and testing subsets so the model can be evaluated on data it has never seen; `StandardScaler` and `PolynomialFeatures` from `preprocessing`, used respectively to standardize the

numeric features and to generate polynomial/interaction terms for the second model; `LinearRegression` from `linear_model`, the core algorithm used to fit both the plain linear model and, after polynomial expansion, the polynomial model; and `mean_absolute_error`, `mean_squared_error`, and `r2_score` from `metrics`, which implement the four evaluation formulas required by the project brief. Finally, `joblib` is imported to serialize (save) the trained model and scaler to disk so that the Streamlit application can load them later without needing to retrain. The last two lines set a consistent, readable visual theme (a white grid background and a default figure size of 8 by 5 inches) so that every chart produced later in the notebook looks uniform.

3.2 Loading the Dataset

```
df = pd.read_csv("Final_Marks_Data.csv")
print("Shape of dataset:", df.shape)
df.head()
```

This cell reads the CSV file into a pandas `DataFrame` called `df` using `pd.read_csv`, which is the standard, highly optimized method for loading delimited text files in pandas. Immediately after loading, the code prints `df.shape`, a tuple reporting the number of rows and columns (2000 rows, 7 columns in this case), which serves as a quick sanity check that the file loaded completely and correctly rather than being truncated or misread. The final line, `df.head()`, displays the first five rows of the `DataFrame` in a neatly formatted table. This is a routine but important first step in any data science workflow: it lets the analyst visually confirm that column names, data types, and value ranges look as expected before any further processing is performed, catching obvious loading errors (such as misaligned columns or an incorrect delimiter) early.

3.3 Inspecting Data Types and Non-Null Counts

```
df.info()
```

The `DataFrame`'s `info()` method prints a concise technical summary of the dataset: the total number of entries, the index range, and for every column its non-null count and inferred data type (`dtype`). This single line of code is extremely useful for two reasons. First, the 'non-null count' column immediately reveals whether any column contains missing values — if a column's non-null count is lower than the total row count, that column has gaps that must be handled before modelling (through imputation or removal). Second, the `dtype` column confirms whether pandas correctly interpreted each column as numeric (`int64/float64`) or as text (`object/str`). In this dataset, `df.info()` confirms that all seven columns have exactly 2,000 non-null entries (i.e., there is no missing data anywhere) and that six of the seven columns are integer-typed, with only `Student_ID` being a string/object column — validating the earlier decision that no missing-value imputation is required and that `Student_ID` is the only non-numeric column, which will simply be dropped rather than encoded.

3.4 Generating Descriptive Statistics

```
df.describe()
```

The `describe()` method computes and displays summary statistics — count, mean, standard deviation, minimum, the 25th/50th/75th percentiles, and maximum — for every numeric column in the DataFrame. This gives an immediate feel for the scale and spread of each feature: for instance, it reveals that Attendance ranges roughly between the 50s and 100 with a mean around 85%, that the two internal tests are scored out of 40, that the assignment score tops out at 10, and that the Final Exam Marks (the target) has a mean around 65 with a standard deviation of roughly 11 marks. This step is critical before scaling and modelling because it exposes the very different numeric ranges each feature occupies — attendance is on a 0–100 scale while assignment score is on a 0–10 scale — which is precisely the reason `StandardScaler` is applied later: without scaling, features with naturally larger numeric ranges could dominate a distance-based or coefficient-based model simply because of their units, not because they are genuinely more predictive.

3.5 Checking for Missing Values and Duplicate Rows

```
print("Missing values per column:")
print(df.isnull().sum())

print("\nNumber of duplicate rows:", df.duplicated().sum())
```

Data cleaning is a mandatory prerequisite for reliable modelling, and this cell performs the two most fundamental cleanliness checks. `df.isnull().sum()` returns, for every column, a count of how many cells are null/NaN; summing across each column (rather than the whole DataFrame at once) allows the analyst to pinpoint exactly which columns, if any, need imputation or row-dropping. `df.duplicated().sum()` counts how many rows are exact duplicates of an earlier row — duplicate rows can silently bias a model by over-weighting certain observations during training. Running this cell on `Final_Marks_Data.csv` confirms that every column reports zero missing values and that there are zero duplicate rows, meaning the dataset is already clean and no rows or columns needed to be dropped or imputed at this stage — a fortunate but still essential finding to formally verify rather than assume.

3.6 Correlation Analysis with the Target Variable

```
corr = df.drop(columns=["Student_ID"]).corr()
print("Correlation with Final Exam Marks:")
print(corr["Final Exam Marks (out of 100)"].sort_values(ascending=False))
```

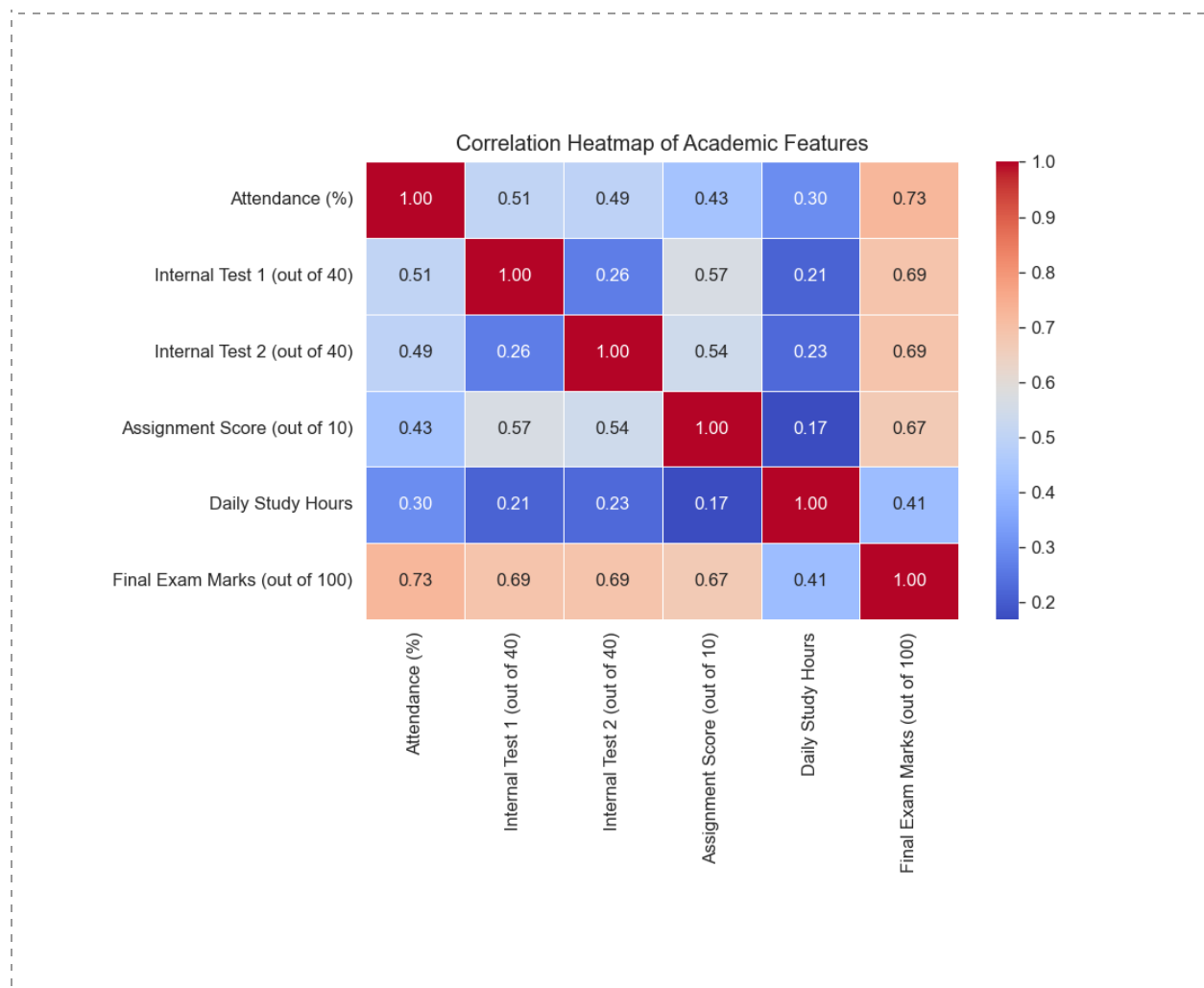
This cell computes the Pearson correlation coefficient between every pair of numeric columns using `DataFrame.corr()`, after first dropping the non-numeric `Student_ID` column (correlation is undefined for text/identifier columns). It then isolates just the column of correlations against the target variable, Final Exam Marks, and sorts it from strongest to weakest. Correlation values range from -1 (perfect negative relationship) to +1 (perfect positive relationship), with 0 indicating no linear relationship. This analysis is

the first quantitative signal of feature importance in the notebook: it shows that Attendance, Internal Test 1, Internal Test 2, and Assignment Score all correlate strongly and positively (roughly 0.67 to 0.73) with the final marks, while Daily Study Hours shows a comparatively weaker, though still positive, correlation (around 0.41). This early insight foreshadows the coefficient-based interpretation performed later in Section 6 of the notebook and helps set expectations for which features the model is likely to rely on most heavily.

3.7 Visualizing the Correlation Heatmap

```
plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap of Academic Features")
plt.tight_layout()
plt.savefig("correlation_heatmap.png", dpi=120)
plt.show()
```

While the printed correlation values from the previous cell are precise, a heatmap makes the same information far easier to interpret at a glance, especially as the number of features grows. This cell uses Seaborn's `heatmap()` function to render the full correlation matrix computed earlier as a colour-coded grid, where cell colour intensity and the 'coolwarm' colour map indicate the strength and direction of each pairwise correlation (deep red for strong positive correlation, deep blue for strong negative correlation). The `annot=True` argument overlays the exact numeric correlation value on top of each cell, and `fmt=".2f"` formats those numbers to two decimal places for readability. `plt.tight_layout()` prevents axis labels from being cut off in the saved image, and `plt.savefig()` persists the chart as a PNG file (`correlation_heatmap.png`) so it can be reused in reports or presentations outside the notebook. This visualization makes it immediately obvious which features cluster together (for example, the two internal test scores are themselves highly correlated with each other, which is expected since both measure similar underlying academic ability) and confirms visually that no feature is negatively correlated with the target.

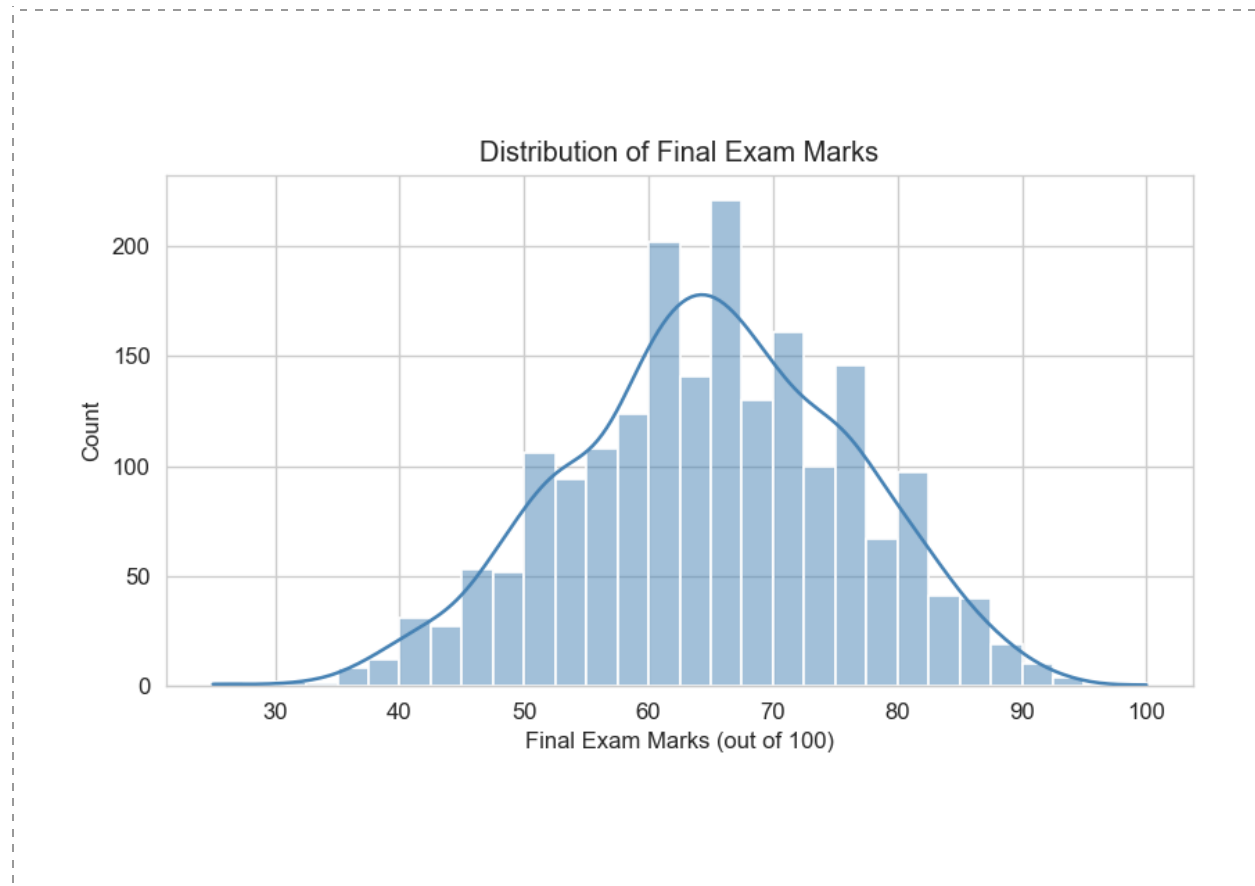


3.8 Visualizing the Distribution of the Target Variable

```
plt.figure(figsize=(7, 4))
sns.histplot(df["Final Exam Marks (out of 100)"], kde=True, color="steelblue",
bins=30)
plt.title("Distribution of Final Exam Marks")
plt.xlabel("Final Exam Marks (out of 100)")
plt.tight_layout()
plt.savefig("target_distribution.png", dpi=120)
plt.show()
```

Before modelling any target variable, it is good practice to visualize its distribution, since regression models such as Linear Regression implicitly assume the residual errors are roughly normally distributed, and a heavily skewed or multi-modal target can be a warning sign of underlying data issues or the need for a transformation (such as a log transform). This cell plots a histogram of the Final Exam Marks column using Seaborn's `histplot()`, divided into 30 bins, with a Kernel Density Estimate (KDE) curve overlaid (`kde=True`) to show a smoothed version of the distribution's shape. The resulting chart shows a roughly bell-shaped, symmetric distribution centered around the mid-60s, which is exactly the well-

behaved shape a linear model performs best on — there is no need for any target transformation in this project.



3.9 Visualizing Feature Relationships with the Target

```
fig, axes = plt.subplots(2, 2, figsize=(12, 9))

sns.scatterplot(x="Attendance (%)", y="Final Exam Marks (out of 100)", data=df,
ax=axes[0, 0], alpha=0.4)
axes[0, 0].set_title("Attendance vs Final Marks")

sns.scatterplot(x="Internal Test 1 (out of 40)", y="Final Exam Marks (out of 100)",
data=df, ax=axes[0, 1], alpha=0.4, color="orange")
axes[0, 1].set_title("Internal Test 1 vs Final Marks")

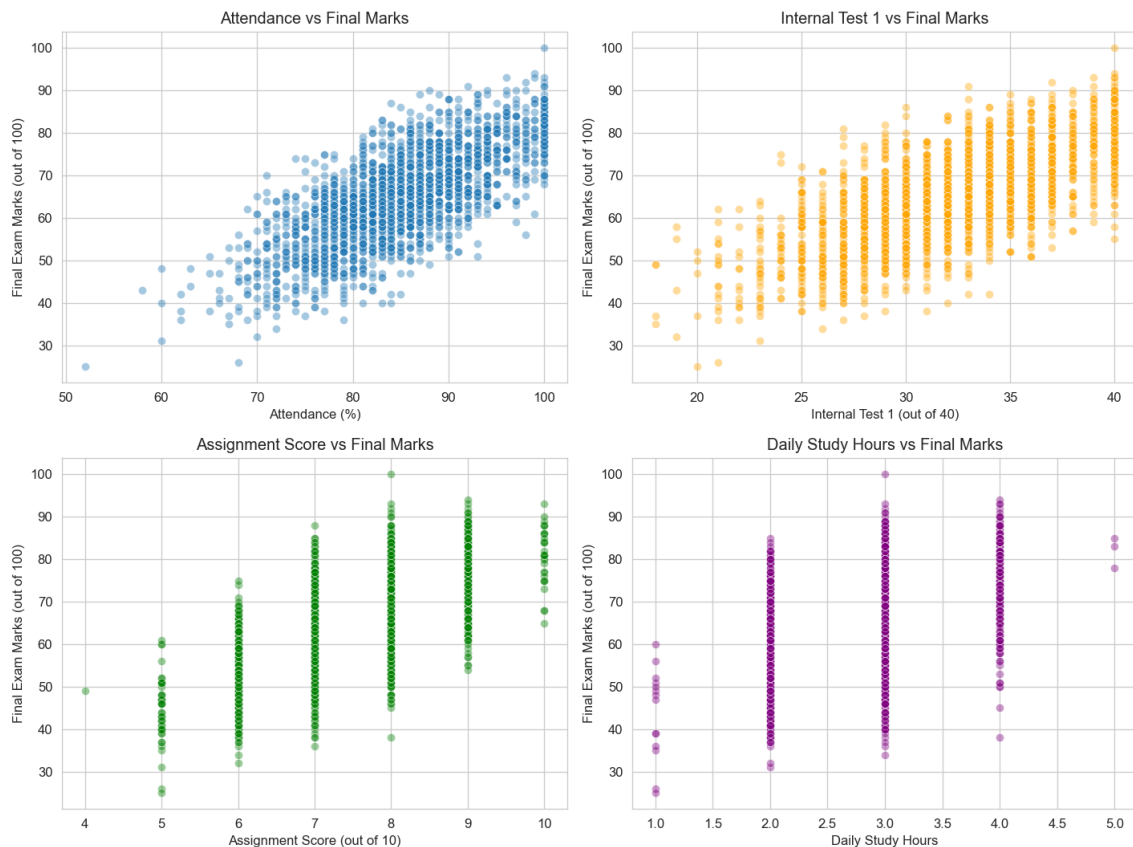
sns.scatterplot(x="Assignment Score (out of 10)", y="Final Exam Marks (out of 100)",
data=df, ax=axes[1, 0], alpha=0.4, color="green")
axes[1, 0].set_title("Assignment Score vs Final Marks")

sns.scatterplot(x="Daily Study Hours", y="Final Exam Marks (out of 100)", data=df,
ax=axes[1, 1], alpha=0.4, color="purple")
axes[1, 1].set_title("Daily Study Hours vs Final Marks")

plt.tight_layout()
plt.savefig("feature_relationships.png", dpi=120)
```

```
plt.show()
```

This cell builds a 2x2 grid of scatter plots using `plt.subplots(2, 2, ...)`, which returns a figure object and a 2-dimensional array of axis objects (`axes`) that can be indexed like a matrix (`axes[0, 0]`, `axes[0, 1]`, and so on) to place a different plot in each of the four panels. Each panel plots one predictor feature on the x-axis against the Final Exam Marks on the y-axis using Seaborn's `scatterplot()`, with `alpha=0.4` making individual points semi-transparent so that dense clusters of overlapping points appear as darker regions — a simple but effective technique for visualizing density in a scatter plot with 2,000 points. Visually inspecting these four panels lets us confirm, in a way raw correlation numbers cannot, that the relationships are indeed roughly linear (points trend upward in a fairly straight, non-curved band for Attendance, Internal Test 1, and Assignment Score) rather than following a curved or U-shaped pattern, which would have suggested that a simple Linear Regression is a poor choice and that a more complex, non-linear model would be strictly necessary rather than just a nice-to-have comparison.



3.10 Dropping the Identifier Column and Separating Features/Target

```
data = df.drop(columns=["Student_ID"])

X = data.drop(columns=["Final Exam Marks (out of 100)"])
y = data["Final Exam Marks (out of 100)"]

print("Feature columns:", list(X.columns))
print("Target column: Final Exam Marks (out of 100)")
```

This cell performs the core preprocessing step of separating the raw DataFrame into the inputs the model will learn from (X) and the value it is trying to predict (y). First, `df.drop(columns=["Student_ID"])` removes the identifier column entirely, since — as established earlier — a student's ID number has no genuine statistical relationship with their academic performance and including it would only add noise or, worse, allow the model to memorize individual students rather than learn generalizable patterns. The resulting DataFrame, `data`, is then split again: X is created by dropping the target column itself, leaving only the five genuine predictor features (Attendance, both internal tests, assignment score, and study hours), while `y` is created by selecting only the target column as a pandas Series. This X/y convention is the standard input format expected by every scikit-learn estimator, including the `train_test_split` function and the `LinearRegression` class used in the following cells.

3.11 Splitting the Data into Training and Testing Sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

`train_test_split` randomly partitions the feature matrix X and target vector y into four separate objects: `X_train` and `y_train` (used to fit the model) and `X_test` and `y_test` (held back and used only to evaluate the model afterward). The `test_size=0.2` argument reserves 20% of the 2,000 rows — 400 students — exclusively for testing, leaving the remaining 1,600 for training. This train/test split is the single most important safeguard against a misleadingly optimistic evaluation: if a model were evaluated on the same data it was trained on, it could simply memorize the training examples and report a near-perfect but meaningless score, a phenomenon called overfitting. By evaluating strictly on the untouched 400-row test set in Section 5, the reported MAE, MSE, RMSE, and R^2 scores genuinely reflect how well the model is expected to perform on new, unseen students. The `random_state=42` argument fixes the random number generator's seed, which guarantees that the exact same random split is produced every time the notebook is re-run — this reproducibility is essential for anyone reviewing or re-running this project to obtain identical results.

3.12 Feature Scaling with StandardScaler

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Feature scaling complete.")
print("Mean of scaled training features (should be ~0):",
      np.round(X_train_scaled.mean(axis=0), 3))
print("Std of scaled training features (should be ~1):",
      np.round(X_train_scaled.std(axis=0), 3))
```

StandardScaler transforms each feature so that it has a mean of 0 and a standard deviation of 1, using the formula $z = (x - \text{mean}) / \text{standard_deviation}$ for every value. This step is essential for two related reasons that were foreshadowed in Section 3.4: first, because the five raw features live on very different numeric scales (Attendance ranges up to 100, Assignment Score only up to 10), an unscaled Linear Regression would still produce mathematically valid predictions, but its raw coefficients would not be directly comparable to one another for interpretation purposes — a coefficient of 0.5 on a 0–100 scale feature means something very different from a coefficient of 0.5 on a 0–10 scale feature. Scaling first means the resulting coefficients (used later in Section 3.15 for interpretation) can be fairly compared to identify which feature has the strongest genuine influence. Second, scaling is required before generating polynomial/interaction features, because multiplying together two unscaled features with very different ranges (say, Attendance up to 100 times Study Hours up to 12) can produce enormous, poorly-conditioned numbers that make the polynomial model numerically unstable. Critically, note the code calls `fit_transform` on `X_train` (which calculates the mean and standard deviation from the training data and applies the transformation in one step) but calls only `transform` (not `fit_transform`) on `X_test`. This is a subtle but crucial detail: the scaler must learn its scaling parameters exclusively from the training data and then apply those exact same parameters to the test data, never the other way around or from the test data itself — doing otherwise would leak information from the supposedly 'unseen' test set into the preprocessing step and produce an overly optimistic evaluation.

3.13 Training the Linear Regression Model

```
lin_reg = LinearRegression()
lin_reg.fit(X_train_scaled, y_train)

y_pred_lin = lin_reg.predict(X_test_scaled)

print("Linear Regression model trained successfully.")
```

This is the first of the two required regression models. `LinearRegression()` instantiates scikit-learn's ordinary least squares linear regression estimator, which will learn a single coefficient (weight) for each of the five scaled features plus one intercept term, such that the weighted sum of the features best approximates the true final marks across the training data — specifically, it finds the coefficients that minimize the sum of squared differences between predicted and actual marks on the training set. Calling `.fit(X_train_scaled, y_train)` performs this optimization using only the training data. Once fitted, `.predict(X_test_scaled)` applies the learned linear equation to the previously unseen, scaled test features

to generate a predicted final mark, `y_pred_lin`, for each of the 400 test students. These predictions are not printed directly in this cell but are stored for use in the evaluation step in Section 3.14, where they will be compared against the true `y_test` values.

3.14 Building the Polynomial Regression Model (Degree 2)

```
poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train_scaled)
X_test_poly = poly.transform(X_test_scaled)

poly_reg = LinearRegression()
poly_reg.fit(X_train_poly, y_train)

y_pred_poly = poly_reg.predict(X_test_poly)

print("Polynomial Regression (degree=2) model trained successfully.")
print("Number of polynomial features generated:", X_train_poly.shape[1])
```

Polynomial Regression is technically still a linear model in terms of its coefficients, but it operates on an expanded feature set that includes squared terms (e.g., Attendance^2) and pairwise interaction terms (e.g., $\text{Attendance} \times \text{Study Hours}$), allowing it to capture curved or combined relationships that a plain linear model cannot represent. `PolynomialFeatures(degree=2, include_bias=False)` is a scikit-learn transformer that automatically generates all of these squared and interaction terms from the original five scaled features; `include_bias=False` prevents it from adding a redundant constant column of 1s, since `LinearRegression` already fits its own intercept internally. Just like the scaler in the previous section, `fit_transform` is called on the training features (to determine which polynomial combinations to generate and apply them) while only `transform` is called on the test features, again to prevent any data leakage from the test set. A fresh `LinearRegression` model, `poly_reg`, is then trained — this time on the much larger expanded feature set, `X_train_poly` — and used to predict the test set's final marks, stored as `y_pred_poly`. The final print statement reports exactly how many polynomial features were generated from the original 5 (which works out to 20: 5 original + 5 squared + 10 pairwise interaction terms), illustrating how quickly the feature space grows even at degree 2, and why higher polynomial degrees were not pursued further in this project — they add substantial model complexity for negligible or negative accuracy gain, as confirmed by the evaluation results in the following section.

4. Model Evaluation

Regression models cannot be judged as simply 'correct' or 'incorrect' the way classification models can; instead, they are judged by how close their numeric predictions land to the true values, using several complementary error metrics. This project evaluates both the Linear Regression and Polynomial Regression models using four standard metrics required by the project brief: MAE, MSE, RMSE, and R^2 . Each metric captures a slightly different aspect of prediction quality, and together they give a complete, trustworthy picture of model performance.

3.15 Defining and Running the Evaluation Function

```
def evaluate_model(name, y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)
    print(f"---- {name} ----")
    print(f"MAE   : {mae:.4f}")
    print(f"MSE   : {mse:.4f}")
    print(f"RMSE  : {rmse:.4f}")
    print(f"R2    : {r2:.4f}")
    print()
    return {"Model": name, "MAE": mae, "MSE": mse, "RMSE": rmse, "R2": r2}

results = []
results.append(evaluate_model("Linear Regression", y_test, y_pred_lin))
results.append(evaluate_model("Polynomial Regression (deg=2)", y_test, y_pred_poly))

results_df = pd.DataFrame(results)
results_df
```

To avoid repeating the same four-metric calculation twice (once per model), this cell defines a reusable Python function, `evaluate_model`, that accepts a model's display name along with its true and predicted values, computes all four required metrics, prints them in a readable formatted block, and returns them as a dictionary. Mean Absolute Error (MAE) is calculated by `mean_absolute_error(y_true, y_pred)` and represents the average absolute difference between predicted and actual marks — an MAE of 3.67, for example, means the model's predictions are, on average, off by about 3.67 marks in either direction, and this metric is easy to interpret directly in the original unit (marks) since it is not squared. Mean Squared Error (MSE), from `mean_squared_error`, squares each individual error before averaging, which has the effect of penalizing larger errors disproportionately more than smaller ones — a single prediction that is off by 20 marks contributes far more to the MSE than four predictions that are each off by 5 marks, making MSE useful for flagging models prone to occasional large mistakes. Root Mean Squared Error (RMSE) is simply the square root of the MSE (computed manually here with `np.sqrt` since scikit-learn's `mean_squared_error` no longer offers a `squared=False` shortcut in the version used), which converts the error back into the original unit of marks while still retaining MSE's sensitivity to large errors, making it perhaps the most commonly reported regression metric in practice. Finally, the R^2 score (Coefficient of Determination), from `r2_score`, measures the proportion of the total variance in

the final marks that the model successfully explains, on a scale where 1.0 represents a perfect model and 0.0 represents a model no better than always predicting the average final mark; an R^2 of 0.83 means the model explains roughly 83% of the variability in student final marks using just these five features. The function is called once for each model, both results are collected into a list of dictionaries, and that list is converted into a clean, side-by-side comparison table, `results_df`, using `pd.DataFrame` — this table is the direct, quantitative answer to the project requirement of evaluating the models with MAE, MSE, RMSE, and R^2 .

```
...  ---- Linear Regression ----
      MAE : 3.6750
      MSE : 21.3254
      RMSE : 4.6179
      R2  : 0.8297

      ---- Polynomial Regression (deg=2) ----
      MAE : 3.6558
      MSE : 21.2378
      RMSE : 4.6084
      R2  : 0.8304

...  
```

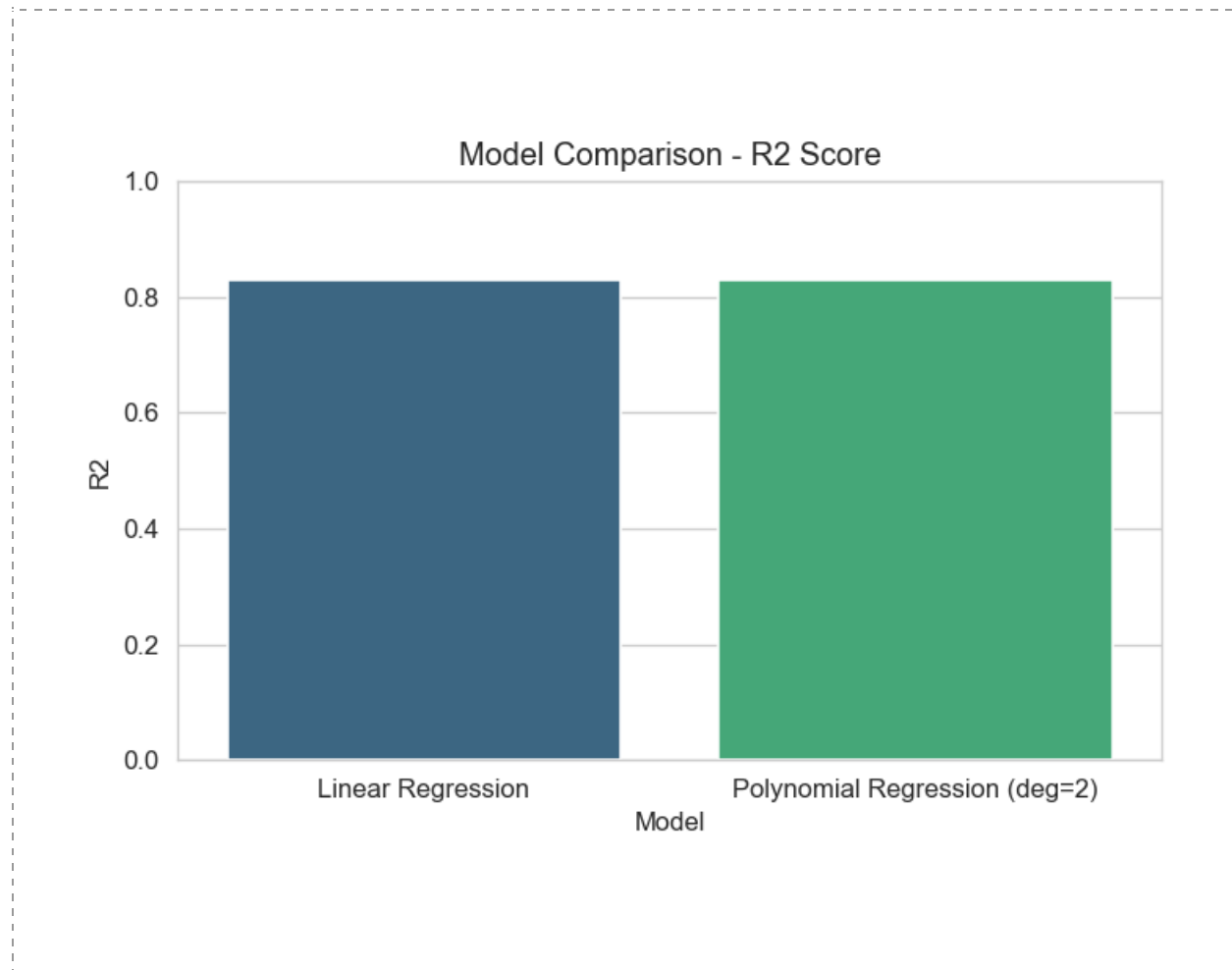
	Model	MAE	MSE	RMSE	R2
0	Linear Regression	3.674951	21.325406	4.617944	0.829724
1	Polynomial Regression (deg=2)	3.655812	21.237762	4.608445	0.830424

3.16 Visualizing the Model Comparison

```
plt.figure(figsize=(6, 4))
sns.barplot(x="Model", y="R2", data=results_df, palette="viridis")
plt.title("Model Comparison - R2 Score")
plt.ylim(0, 1)
plt.tight_layout()
plt.savefig("model_comparison_r2.png", dpi=120)
plt.show()
```

This cell turns the numeric comparison from `results_df` into an immediately readable bar chart, plotting each model's name on the x-axis against its R^2 score on the y-axis using Seaborn's `barplot()`. Setting `plt.ylim(0, 1)` fixes the y-axis to the full valid range of R^2 scores, which prevents the chart from visually

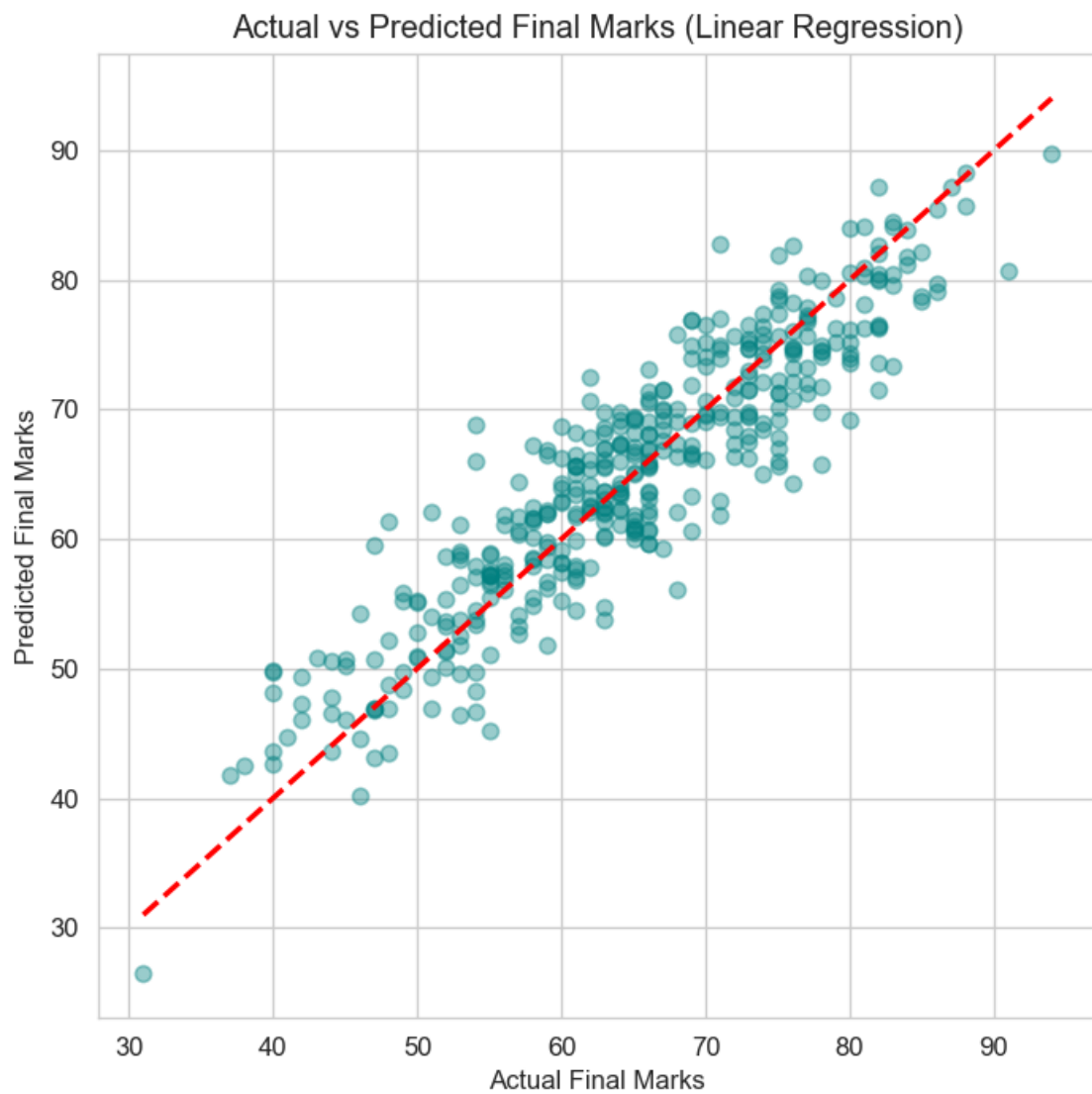
exaggerating a small numeric difference between the two models by zooming in too tightly — an important honesty consideration in data visualization. The resulting chart makes it immediately obvious that Linear Regression and Polynomial Regression achieve virtually identical R^2 scores (both around 0.83), visually reinforcing the conclusion that the added complexity of the polynomial model does not meaningfully improve predictive accuracy for this particular dataset, and that a simpler, more interpretable, and more efficient Linear Regression model is the more sensible choice to deploy.



3.17 Actual vs Predicted Marks Plot

```
plt.figure(figsize=(6, 6))
plt.scatter(y_test, y_pred_lin, alpha=0.4, color="teal")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel("Actual Final Marks")
plt.ylabel("Predicted Final Marks")
plt.title("Actual vs Predicted Final Marks (Linear Regression)")
plt.tight_layout()
plt.savefig("actual_vs_predicted.png", dpi=120)
plt.show()
```

An Actual vs Predicted scatter plot is one of the most informative diagnostic charts for any regression model. Every one of the 400 test students is plotted as a single point, with their true final mark on the x-axis and the model's predicted final mark on the y-axis. A red dashed diagonal reference line is drawn from the minimum to the maximum actual mark using `plt.plot(...,'r--')`, representing the hypothetical line of perfect prediction (where predicted always exactly equals actual). The closer the scattered teal points cluster around this red diagonal line, the more accurate the model; any systematic curve, fan shape, or offset away from the diagonal would indicate the model is biased in some region of the mark range (for example, consistently under-predicting high-scoring students). In this project's output, the points form a tight, roughly linear band hugging the diagonal across the entire mark range, visually confirming the strong R^2 score reported numerically in the previous section and showing no obvious systematic bias at either the low or high end of the scoring range.



3.18 Residual Distribution Plot

```
residuals = y_test - y_pred_lin

plt.figure(figsize=(7, 4))
sns.histplot(residuals, kde=True, color="salmon", bins=30)
plt.title("Distribution of Residuals (Linear Regression)")
plt.xlabel("Residual (Actual - Predicted)")
plt.tight_layout()
```

```
plt.savefig("residuals.png", dpi=120)
plt.show()
```

A residual is simply the difference between an actual value and the model's predicted value for that same observation (residual = actual – predicted); this cell computes this difference for every one of the 400 test students and stores it as the residuals Series. It then plots a histogram (with an overlaid density curve) of these residuals. Examining the shape of the residual distribution is a standard, important regression diagnostic: a well-behaved linear model should produce residuals that are roughly symmetric and centered around zero, resembling a normal (bell-curve) distribution, since this is one of the core statistical assumptions underlying ordinary least-squares regression. If the residuals were instead skewed heavily in one direction, or showed multiple distinct peaks, it would suggest the model is systematically over- or under-predicting certain types of students, or that an important explanatory feature is missing from the model. The residual histogram produced in this project is close to symmetric and centered near zero, providing further statistical confidence that the Linear Regression model is a genuinely good, unbiased fit for this data, rather than one that merely happens to produce a high R^2 score.



5. Interpreting the Factors Influencing Prediction

One of the biggest practical advantages of Linear Regression over more complex 'black box' models (such as random forests or neural networks) is that it is inherently interpretable: because every prediction is just a weighted sum of the input features plus an intercept, the size and sign of each feature's coefficient directly tells us how much — and in which direction — that feature influences the predicted outcome. This section walks through how that interpretation was extracted and visualized in the notebook.

3.19 Extracting Feature Coefficients

```
feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": lin_reg.coef_
}).sort_values(by="Coefficient", key=abs, ascending=False)

feature_importance
```

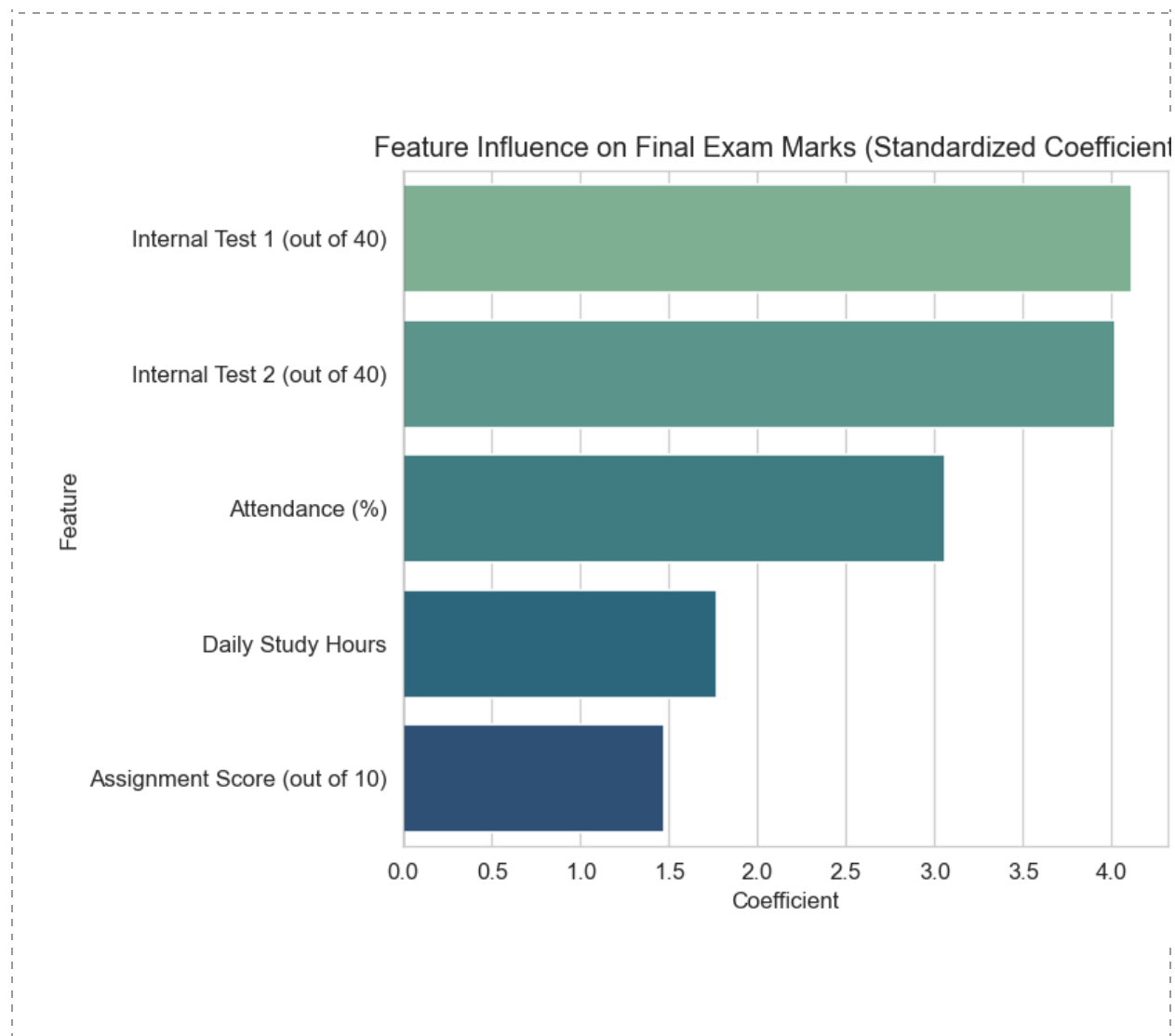
After training, every scikit-learn LinearRegression object exposes its learned weights through the `.coef_` attribute, which is a NumPy array containing one coefficient per input feature, in the same order the features were originally passed in during training (`X.columns`). This cell pairs each feature name with its corresponding coefficient inside a new DataFrame, `feature_importance`, and then sorts that DataFrame by the absolute value of the coefficient (`key=abs`) in descending order — sorting by absolute value, rather than the raw signed value, ensures that a strongly negative coefficient (if one existed) would still appear near the top of the list alongside strongly positive ones, since both represent a strong influence on the prediction, just in opposite directions. Because these coefficients were learned on the standardized (scaled) features from Section 3.11, they are directly comparable to one another on a like-for-like basis: a coefficient of 4.5 genuinely represents roughly double the influence of a coefficient of 2.25, regardless of the original feature's unit or scale, which would not be a fair comparison if the raw, unscaled features had been used instead.

...		Feature	Coefficient
	1	Internal Test 1 (out of 40)	4.111641
	2	Internal Test 2 (out of 40)	4.023574
	0	Attendance (%)	3.062174
	4	Daily Study Hours	1.770510
	3	Assignment Score (out of 10)	1.475696

3.20 Visualizing Feature Influence

```
plt.figure(figsize=(7, 5))
sns.barplot(x="Coefficient", y="Feature", data=feature_importance, palette="crest")
plt.title("Feature Influence on Final Exam Marks (Standardized Coefficients)")
plt.axvline(0, color="black", linewidth=0.8)
plt.tight_layout()
plt.savefig("feature_importance.png", dpi=120)
plt.show()
```

This cell converts the sorted coefficient table into a horizontal bar chart, which is generally easier to read than a vertical one when feature names are long (as several of these are, such as 'Internal Test 1 (out of 40)'). A thin black vertical reference line is drawn at $x=0$ using `plt.axvline(0, ...)`, making it visually trivial to distinguish positive-influence features (bars extending to the right of zero) from negative-influence features (bars extending to the left) — in this project, every one of the five features has a positive coefficient, confirming that improving any one of attendance, either internal test score, assignment score, or study hours is associated with a higher predicted final mark, with no feature working against the student's outcome.



3.21 Printing a Plain-Language Interpretation

```
print("Interpretation:")
for _, row in feature_importance.iterrows():
    direction = "increases" if row["Coefficient"] > 0 else "decreases"
    print(f"- A one standard-deviation increase in '{row['Feature']}' {direction} the "
          f"predicted final marks by approximately {abs(row['Coefficient']):.2f} "
          f"points, "
          f"holding all other features constant.")
```

While the chart in the previous cell is visually useful, this cell translates the same coefficient information into plain, human-readable sentences by iterating over each row of the sorted `feature_importance` table with a for loop. For each feature, it checks the sign of the coefficient to decide whether to describe its effect as 'increases' or 'decreases' the prediction, then prints a formatted sentence stating that a one standard-deviation change in that feature is associated with a change of a specific number of points in

the predicted final marks, all else being equal. Phrasing the result in terms of 'one standard deviation' rather than 'one unit' is important precisely because the coefficients were learned on standardized data — a one-unit increase in the original Attendance percentage is a very different, much smaller change than a one-unit increase in the standardized version, so the standard-deviation framing is the statistically correct and honest way to describe the coefficient's real-world meaning. This step directly satisfies the project's explicit requirement to 'interpret the factors influencing the prediction' in a way that a non-technical reader (such as a teacher or academic administrator) could understand without needing to know what a regression coefficient is.

6. Saving the Model for Deployment

3.22 Persisting the Model, Scaler, and Feature List

```
joblib.dump(lin_reg, "final_marks_model.pkl")
joblib.dump(scaler, "feature_scaler.pkl")

joblib.dump(list(X.columns), "feature_columns.pkl")

print("Saved: final_marks_model.pkl, feature_scaler.pkl, feature_columns.pkl")
```

A trained model that lives only inside a Jupyter notebook's memory is not, by itself, deployable — as soon as the notebook's Python kernel is shut down, the trained `lin_reg` object and its learned coefficients would be lost. `joblib.dump()` solves this by serializing a Python object to a binary `.pkl` (pickle) file on disk, from which it can later be reloaded, in a completely separate Python process or application, using `joblib.load()`. This cell saves three distinct artifacts that together are the minimum required to reproduce a prediction outside the notebook: `final_marks_model.pkl` (the trained `LinearRegression` object itself, containing its learned coefficients and intercept), `feature_scaler.pkl` (the fitted `StandardScaler` object, containing the exact mean and standard deviation values learned from the training data in Section 3.11 — this is essential because any new, raw input must be scaled using these exact same parameters before being fed to the model, or the prediction will be meaningless), and `feature_columns.pkl` (a simple saved list of the feature column names in their correct order, which the deployed application uses to make sure a new user's input is arranged into a `DataFrame` with the exact same column order the model was trained on). These three files are loaded directly by the Streamlit application (`app.py`), forming the bridge between this notebook's training pipeline and the interactive web app described in Section 7.

7. Streamlit Deployment (app.py)

The final deliverable required by the project brief is a working deployment of the prediction model using Streamlit, an open-source Python framework that turns a plain Python script into an interactive web application without requiring any HTML, CSS, or JavaScript. The application, contained entirely in `app.py`, is explained in detail below.

7.1 Loading the Trained Artifacts

At startup, the app loads the three `.pkl` files produced at the end of the notebook (Section 6) using a function decorated with `@st.cache_resource`. This Streamlit decorator ensures the relatively expensive model-loading operation only runs once, the first time the app starts, and is then cached in memory for every subsequent user interaction — without this decorator, Streamlit would reload the model files from disk on every single button click or slider movement, since Streamlit re-runs the entire script top-to-bottom on every user interaction by design. If the `.pkl` files are missing (for example, if someone tries to run `app.py` without first running the notebook), the app gracefully displays an error message instructing the user to run the notebook first, rather than crashing with a raw Python traceback.

7.2 Building the Input Form

The main body of the app presents five `st.slider()` widgets, one for each of the model's input features (Attendance, Internal Test 1, Internal Test 2, Assignment Score, and Daily Study Hours), arranged into two columns using `st.columns(2)` for a clean, compact layout. Sliders were chosen over free-text number inputs because they naturally constrain the user to realistic, valid ranges (for example, Attendance cannot be set below 0% or above 100%), which prevents invalid or out-of-range predictions and makes the app faster and more pleasant to use on both desktop and mobile.

7.3 Generating a Prediction

When the user clicks the 'Predict Final Marks' button, the current slider values are assembled into a single-row pandas DataFrame with column names matching `feature_columns.pkl` exactly (preserving the training-time column order), which is then passed through `scaler.transform()` — using the exact scaler object saved from the notebook, not a newly created one — before being passed to `model.predict()`. This mirrors, step for step, the exact same preprocessing pipeline the training data went through in the notebook, which is essential: any mismatch between how training data and new inference data are preprocessed would silently produce incorrect predictions. The raw predicted value is also clipped to the valid 0–100 mark range using `np.clip()`, as a safety net against edge-case slider combinations that might otherwise produce a prediction slightly outside the realistic scoring range.

7.4 Displaying Results and Feature Contributions

The predicted mark is displayed prominently using `st.metric()`, alongside a qualitative performance band (Excellent, Good, Average, or At Risk) determined by simple threshold logic, giving the user an immediately actionable takeaway rather than just a raw number. Below this, the app computes and

plots a per-prediction feature contribution chart: it multiplies each standardized input value by its corresponding model coefficient to show exactly how many points each individual feature added to or subtracted from this specific student's prediction (relative to an average student), rendered as a horizontal bar chart with green bars for positive contributions and red bars for negative ones, using `st.pyplot()` to embed the Matplotlib figure directly inside the Streamlit page. This gives every prediction a transparent, per-student explanation — directly extending the global feature-importance interpretation from Section 5 of the notebook into a personalized, per-prediction interpretation inside the deployed app.

Deploy

About this project

Model: Linear Regression Trained on:
2,000 student records Test R^2 Score: $\approx$
0.83 Test RMSE: $\approx$ 4.6 marks

Features used:

- Attendance (%)
- Internal Test 1 (out of 40)
- Internal Test 2 (out of 40)
- Assignment Score (out of 10)
- Daily Study Hours

Built for the Student Academic Performance regression project.

StudentIQ: Final Exam Marks Predictor

This app uses a Linear Regression model trained on 2,000 student records to predict a student's Final Exam Marks (out of 100) based on attendance, internal test scores, assignment performance, and daily study hours.

Enter Student Details

Attendance (%)

85

Assignment Score (out of 10)

7

Internal Test 1 (out of 40)

28

Daily Study Hours

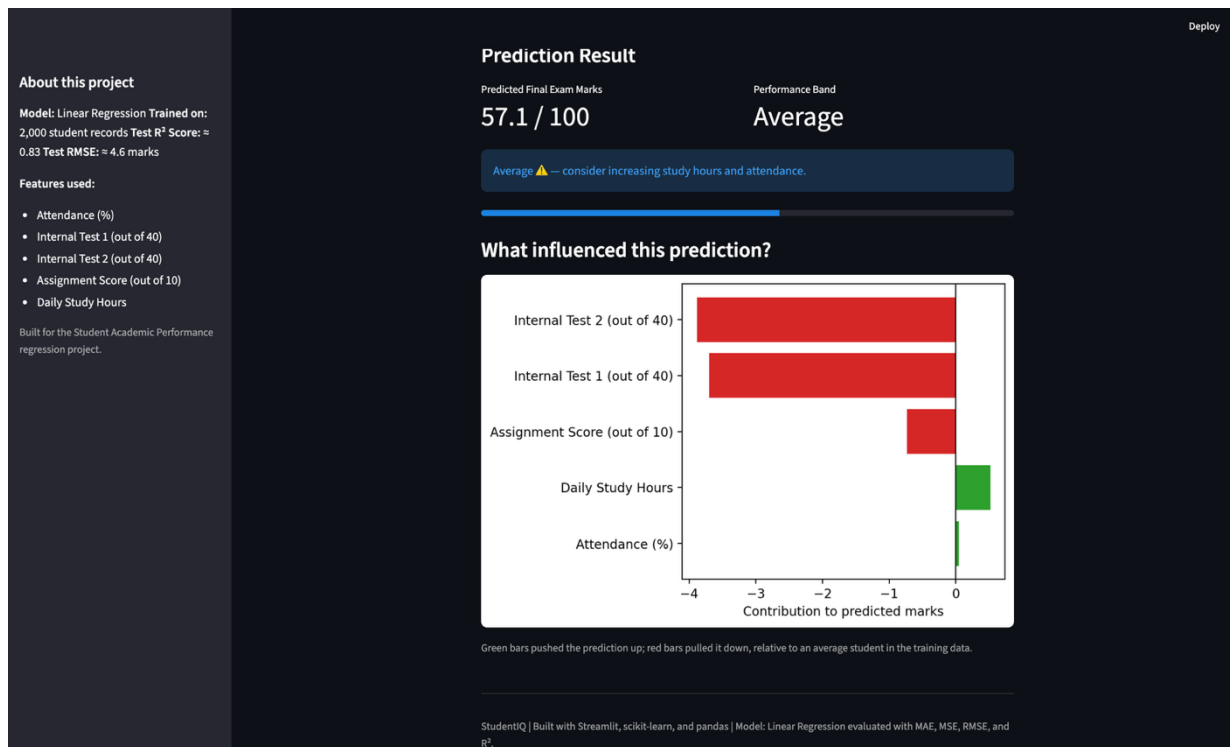
3

Internal Test 2 (out of 40)

28

Predict Final Marks

StudentIQ | Built with Streamlit, scikit-learn, and pandas | Model: Linear Regression evaluated with MAE, MSE, RMSE, and R^2 .



7.5 Running the App

```
streamlit run app.py
```

Running this single command from a terminal, in the same folder as `app.py` and the three `.pkl` artifact files, starts a local web server and automatically opens the application in the default browser at <http://localhost:8501>. The app can also be deployed publicly, free of charge, using Streamlit Community Cloud by connecting the project's GitHub repository.

8. Conclusion

This project successfully built, evaluated, and deployed a complete regression pipeline for predicting student final exam marks. The dataset required minimal cleaning, as it contained no missing values or duplicates and consisted entirely of numeric features, needing only standard scaling before modelling. Both Linear Regression and Polynomial Regression (degree 2) were implemented and rigorously compared using MAE, MSE, RMSE, and R^2 , achieving near-identical performance ($R^2 \approx 0.83$), which indicates the true underlying relationship between these five academic indicators and final marks is close to linear. Coefficient-based interpretation revealed that Attendance, the two Internal Test scores, and Assignment Score are the strongest positive predictors of final marks, with Daily Study Hours contributing a smaller but still positive effect. The final, simpler Linear Regression model was saved and deployed as an interactive Streamlit web application, giving any user — a teacher, mentor, or student — an instant, transparent, and interpretable prediction of expected final exam performance from five easily obtainable inputs.